

ECE 315

Steven Knudsen, PhD, PEng

Lecture 32 :

- RP2040 Stepper Examples
 - L298N H-Bridge
 - A4988 Driver



UNIVERSITY
OF ALBERTA

Last Lecture

- Motion control
- Encoders
- Servo and DC Motors
- Other actuators

This Lecture

- RP2040 Stepper Examples
 - L298N H-Bridge
 - A4988 Driver

Learning Objectives

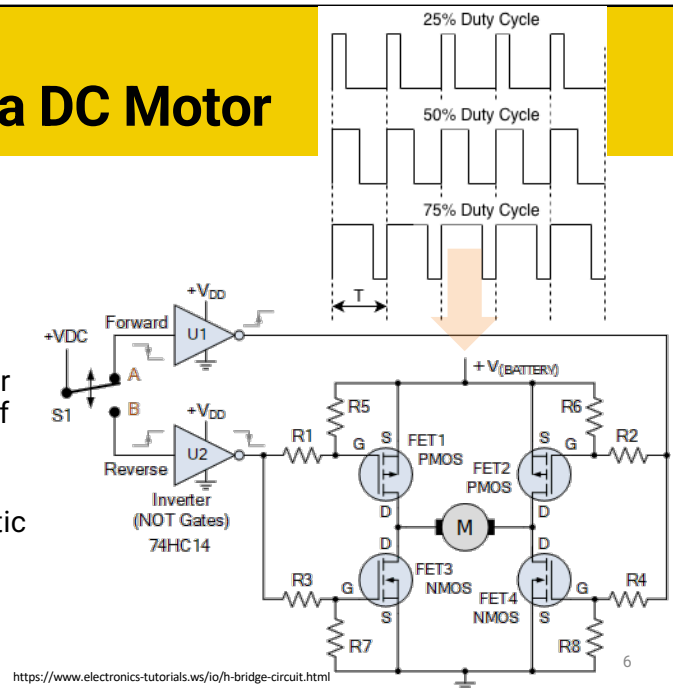
- Explain the operation of an H-bridge circuit and how it enables bidirectional control and speed control (via PWM) of DC motors
- Implement stepper motor control using the L298N, including generating appropriate coil activation sequences for full-step and half-step operation
- Explain how GPIO control and timing parameters affect stepper motor behavior, including direction, speed, and number of steps
- Compare the L298N and A4988 drivers, highlighting differences in control complexity, features (e.g., microstepping), and application use cases
- Use the A4988 driver interface (STEP/DIR and microstepping inputs) to control a stepper motor and configure different stepping resolutions

Plain Old H-Bridge vs Modern Motor Controller

- Look at two popular, inexpensive motor controllers
 - L298N – An H-bridge
 - A4988 – An H-bridge-based controller with extra bells and whistles

H-Bridge Control of a DC Motor

- This circuit illustrates the forward and reverse control via switch S1
- Motor speed is accomplished via pulse-width modulation (PWM)
- As with stepper motors, an encoder will help with closed-loop control of the motor
- Unlike the stepper, the motor position cannot be simply held static

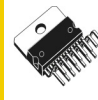


We looked at this last lecture.

One thing to remark on is that the frequency of the PWM signal needs to be significantly higher than the switching signal (S1).

Imagine the switch S1 operates at 2x the PWM duty cycle and is offset by $\frac{1}{4}$ of the PWM frequency. Then one state, one direction through the H-bridge would never be energized

L298 Dual (Full) H-Bridge Driver IC



Multiwatt15 V

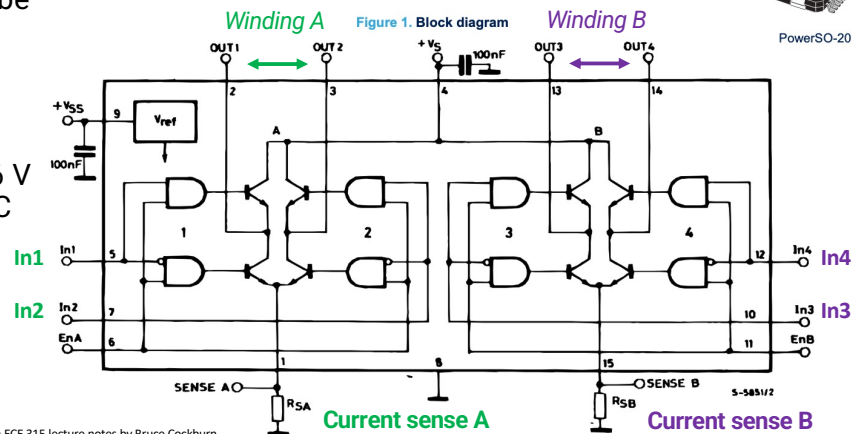


Multiwatt15 H



PowerSO-20

- TTL-compatible control inputs. Can be driven by many microcontrollers
- The H-bridge drives inductive loads (motors) at up to 46 V and a total of 4 A DC

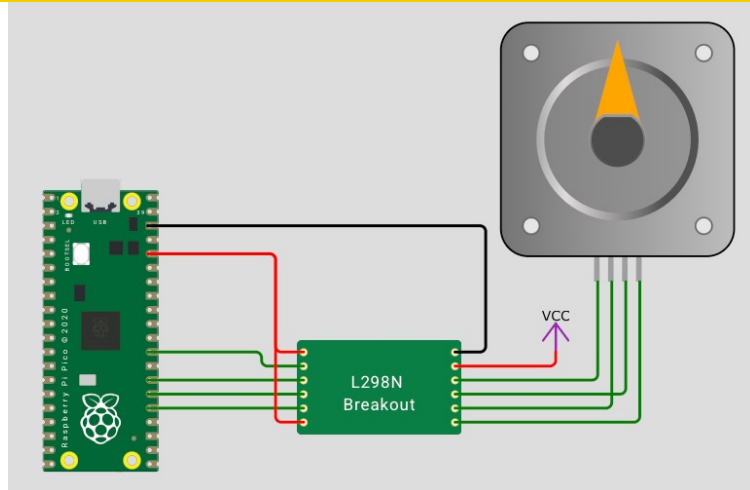


UNIVERSITY
OF ALBERTA

Adapted from ECE 315 lecture notes by Bruce Cockburn
<https://www.st.com/resource/en/datasheet/l298.pdf>

Wokwi Stepper Control – L298N

- Here ENA and ENB are tied high, so the H-bridge is always on
- The Pico outputs the signals [IN1,IN2,IN3,IN4] in a specific pattern to get the motor to step
- It can step FULL or HALF steps in this example



L298N Controller

```
/**
 * ECE 315 Computer Interfacing
 *
 * @file main.c
 * @details Bare-Metal L298N Stepper Motor Driver
 * @author Steven Knudsen
 * @copyright 2025, University of Alberta
 * @version 1.0
 * @licence MIT
 */

#include <stdint.h>
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"

// Rotation definitions
#define CW true // Clockwise
#define CCW false // Counter-clockwise

// GPIO Pin Definitions
#define IN1_PIN 22
#define IN2_PIN 21
#define IN3_PIN 20
#define IN4_PIN 19

// Pin Masks - using a mask allows the state of the pins to
// be set in one instruction
#define IN1_MASK (1u << IN1_PIN)
#define IN2_MASK (1u << IN2_PIN)
#define IN3_MASK (1u << IN3_PIN)
#define IN4_MASK (1u << IN4_PIN)
#define ALL_IN_PINS (IN1_MASK | IN2_MASK | IN3_MASK | IN4_MASK)

// Stepping Sequence structure
typedef struct {
    uint8_t in1;
    uint8_t in2;
    uint8_t in3;
    uint8_t in4;
} StepSequence_t;

// Half-step sequence (8 steps per full step)
static const StepSequence_t HALF_STEP[8] = {
    {1, 0, 0, 1}, // Step 7
    {1, 0, 0, 0}, // Step 0
    {1, 1, 0, 0}, // Step 1
    {0, 1, 0, 0}, // Step 2
    {0, 1, 1, 0}, // Step 3
    {0, 0, 1, 0}, // Step 4
    {0, 0, 1, 1}, // Step 5
    {0, 0, 0, 1}, // Step 6
};
```



9

First we define which pins of the Pico will map to the INx inputs\

The mask is used to turn on or off pins in groups

The stepping sequence structure is just a collection of the INx pins

L298N Controller

```
// Half-step sequence (8 steps per full step)
static const StepSequence_t HALF_STEP[8] = {
    {1, 0, 0, 1}, // Step 7
    {1, 0, 0, 0}, // Step 0
    {1, 1, 0, 0}, // Step 1
    {0, 1, 0, 0}, // Step 2
    {0, 1, 1, 0}, // Step 3
    {0, 0, 1, 0}, // Step 4
    {0, 0, 1, 1}, // Step 5
    {0, 0, 0, 1}, // Step 6
};

// Full-step sequence (4 steps, higher torque)
static const StepSequence_t FULL_STEP[4] = {
    {1, 0, 0, 1}, // Step 0
    {1, 1, 0, 0}, // Step 1
    {0, 1, 1, 0}, // Step 2
    {0, 0, 1, 1}, // Step 3
};

// Global step position
static volatile int32_t step_position_half = 0;
static volatile int32_t step_position_full = 0;

// Forward declarations
void stepper_set_outputs(const StepSequence_t* seq);
void stepper_step_half(bool clockwise, uint32_t steps, uint32_t speed_us);
void stepper_step_full(bool clockwise, uint32_t steps, uint32_t speed_us);

void initialize_gpios(void) {
    // Initialize the INx pins as outputs
    gpio_init(IN1_PIN);
    gpio_init(IN2_PIN);
    gpio_init(IN3_PIN);
    gpio_init(IN4_PIN);
    gpio_set_dir(IN1_PIN, GPIO_OUT);
    gpio_set_dir(IN2_PIN, GPIO_OUT);
    gpio_set_dir(IN3_PIN, GPIO_OUT);
    gpio_set_dir(IN4_PIN, GPIO_OUT);

    // Turn them off
    gpio_put(IN1_PIN, 0);
    gpio_put(IN2_PIN, 0);
    gpio_put(IN3_PIN, 0);
    gpio_put(IN4_PIN, 0);
}
```



10

The Half- and Full-step sequences are defined. They are the pattern of inputs that energize the A and B coils separately and together to cause a half or full step, respectively.

Some convenient functions to manage multiple steps in one direction are defined and the GPIO initialized.

L298N Controller

```
// Set stepper motor outputs
void stepper_set_outputs(const StepSequence_t* seq) {
    uint32_t out_val = 0;

    if (seq->in1) out_val |= IN1_MASK;
    if (seq->in2) out_val |= IN2_MASK;
    if (seq->in3) out_val |= IN3_MASK;
    if (seq->in4) out_val |= IN4_MASK;

    // Set all the GPIOs at once
    gpio_clr_mask(ALL_IN_PINS);
    gpio_set_mask(out_val);
}

// Half-step mode (smoother, 400 steps/rev for 1.8° motor)
void stepper_step_half(bool clockwise, uint32_t steps, uint32_t speed_us) {

    int8_t dir = (clockwise) ? -1 : 1;

    for (uint32_t i = 0; i < steps; i++) {
        step_position_half += dir;
        if (step_position_half < 0) step_position_half = 7;
        if (step_position_half > 7) step_position_half = 0;
        stepper_set_outputs(&HALF_STEP[step_position_half]);
        sleep_us((speed_us));
    }
}

// Full-step mode (higher torque, 200 steps/rev for 1.8° motor)
void stepper_step_full(bool clockwise, uint32_t steps, uint32_t speed_us) {

    int8_t dir = (clockwise) ? -1 : 1;

    for (uint32_t i = 0; i < steps; i++) {
        step_position_full += dir;
        if (step_position_full < 0) step_position_full = 3;
        if (step_position_full > 3) step_position_full = 0;
        stepper_set_outputs(&FULL_STEP[step_position_full]);
        sleep_us((speed_us));
    }
}

// Set the step phase so that it corresponds to the simulated
// stepper motor position zero
void initialize_stepper(void) {
    step_position_half = 7;
    step_position_full = 3;
}
```



11

`stepper_set_outputs` creates a GPIO pin mask based on the input sequence aka pattern. It then uses synchronous IO commands that are implicit in the two `gpio_` commands to turn off briefly all pins and then turn on all those that are needed in this step.

We want to avoid turning them on or off individually for number of practical reasons, probably the most important being that we don't want to overlap signals when they should not be, or create glitches when they should be.

The last function, `initialize_stepper`, ensures that the driver starts in the right place. That is, assuming the motor is positioned in its starting rotation, which we assume represents 0 degrees.

Since we have no encoder or other way to determine position, this is in effect an open-loop initialization.

L298N Controller

- The main program initializes the GPIOs and stepper
- Stepping the motor requires we set the
 - Direction, CW or CCW
 - Number of steps
 - The time in μs between steps
- The time basically sets the speed of the rotation



```
// Main Program
int main(void) {
    stdio_init_all();

    printf("L298N driver example!\n");

    initialize_gpios();
    initialize_stepper();

    // Demo: Continuous rotation
    while (1) {
        // Full-step example (higher torque, less smooth)
        printf("Full-step at 30 rpm\n");
        stepper_step_full(CW, 200, 10000); // full rev. 200 steps * 10 ms/step = 2 s/rev
        stepper_step_full(CW, 200, 10000); // 60 s/min / (2 s/rev) = 30 rpm
        stepper_step_full(CW, 100, 10000); // half rev.
        stepper_step_full(CW, 100, 10000);
        sleep_ms(1000);

        printf("Half-step at 15 rpm\n");
        stepper_step_half(CW, 400, 10000); // full rev. 400 steps * 10 ms/step = 4 s/rev
        stepper_step_half(CW, 400, 10000); // 60 s/min / (4 s/rev) = 15 rpm
        stepper_step_half(CW, 200, 10000); // half rev.
        stepper_step_half(CW, 200, 10000);
        sleep_ms(1000);
    } // while forever

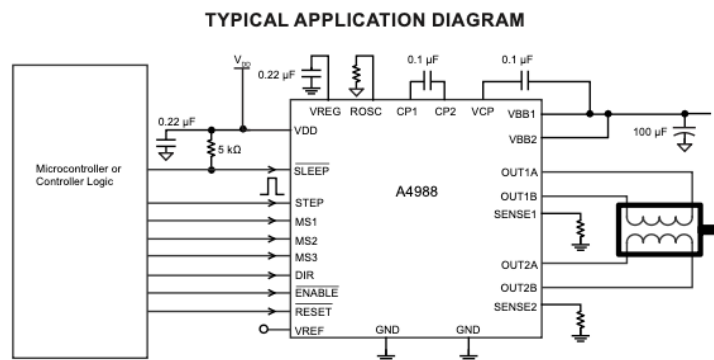
    return 0;
}
```

<https://wokwi.com/projects/450987602193048577>

12

Wokwi Stepper Control – A4988

- The A4988 is a “DMOS Microstepping Driver with Translator and Overcurrent Protection”
- The output looks similar to the L298N
- The input signals are simpler with STEP and DIR being the only two needed to make the motor step



A double-diffused metal-oxide semiconductor (DMOS) is a **circuit structure used in power MOSFETs**.

<https://www.allegromicro.com/en/products/motor-drivers/brush-dc-motor-drivers/a4988> 13



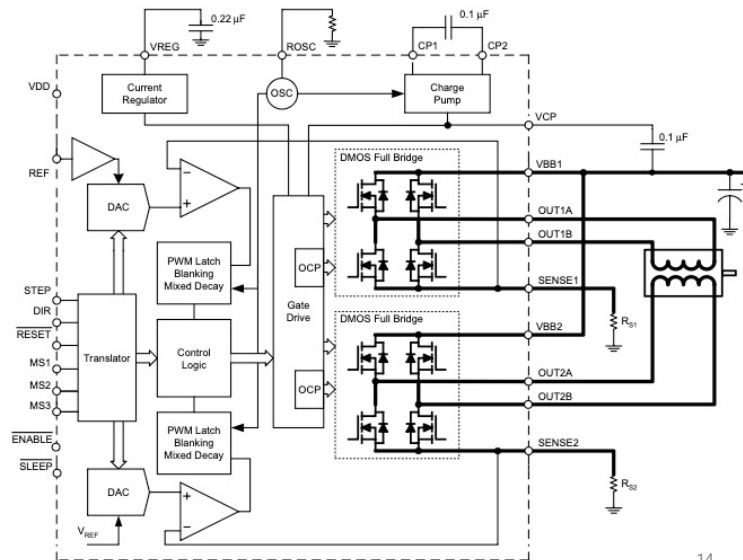
A double-diffused metal-oxide semiconductor (DMOS) is a **circuit structure used in power MOSFETs**.

A4988

- There are two H-bridges as expected
- MS1, MS2, and MS3 set the steps to FULL, $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, or $\frac{1}{16}$
- There is support for PWM and current limiting and over-current protection
 - We won't bother with them today



Functional Block Diagram

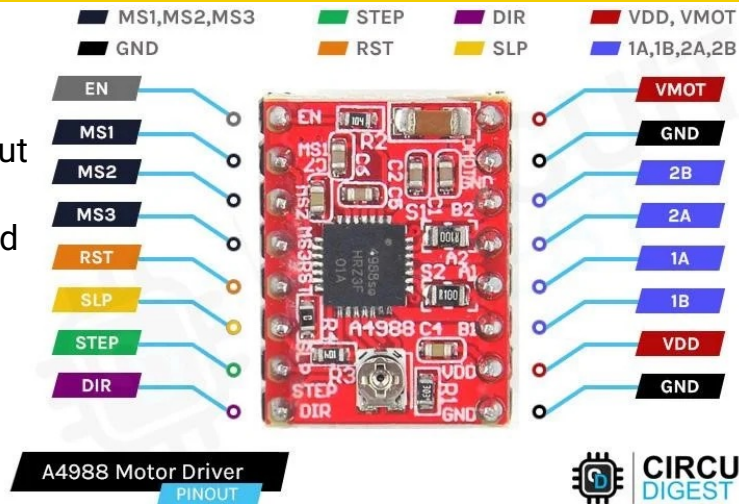


<https://www.allegromicro.com/en/products/motor-drivers/brush-dc-motor-drivers/a4988>

14

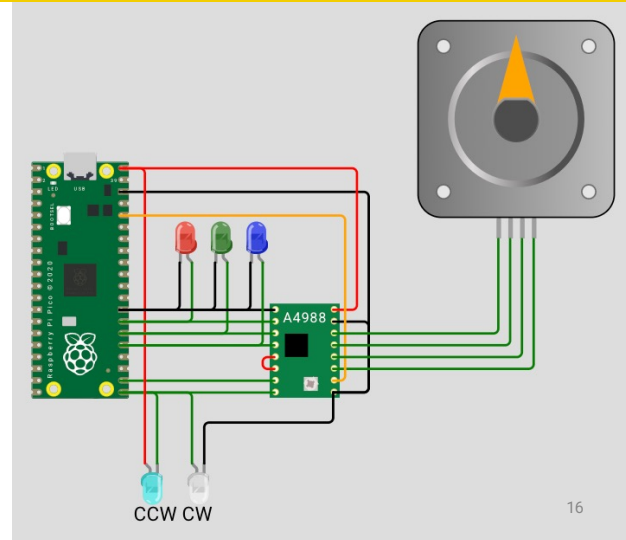
Typical A4988 Module

- The colour is different than in Wokwi, but the pinout are the same
- Some pins are pulled high or low
 - SLEEP is active low and pulled high internally



Wokwi Stepper Control – A4988

- In this example we've added RGB LEDs to indicate the microstepping settings (MS1, MS2, MS3)
- LEDs for direction
- SLEEP is pulled high, so we use it to pull RESET (active low) high as well



16


```

/**
 * ECE 315 Computer Interfacing
 *
 * @file main.c
 * @details Bare-Metal A4988 Motor Driver
 * @author Steven Knudsen
 * @copyright 2025, University of Alberta
 * @version 1.0
 * @licence MIT
 */

#include <stdint.h>
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"

// GPIO Pin Definitions
#define DIR_PIN 16 // Step
#define STP_PIN 17 // Direction
#define MS3_PIN 19 // Microstepping resolution 3
#define MS2_PIN 20 // Microstepping resolution 2
#define MS1_PIN 21 // Microstepping resolution 1

// Microstepping Pin Masks
#define MS1_MASK (1u << MS1_PIN)
#define MS2_MASK (1u << MS2_PIN)
#define MS3_MASK (1u << MS3_PIN)

typedef enum Microstepping_mask {
    FULL_STEP = 0,
    HALF_STEP = MS1_MASK,
    QUARTER_STEP = MS2_MASK,
    EIGHTH_STEP = (MS1_MASK | MS2_MASK),
    SIXTEENTH_STEP = (MS1_MASK | MS2_MASK | MS3_MASK),
    ALL_MS = (MS1_MASK | MS2_MASK | MS3_MASK)
} Microstepping_mask_t;

typedef enum Direction {
    CW,
    CCW
} Direction_t;

void initialize_gpios(void) {
    // All pins are output and used to drive/control the A4988
    gpio_init(STP_PIN);
    gpio_init(DIR_PIN);
    gpio_init(MS3_PIN);
    gpio_init(MS2_PIN);
    gpio_init(MS1_PIN);
    gpio_set_dir(STP_PIN, GPIO_OUT);
    gpio_set_dir(DIR_PIN, GPIO_OUT);
    gpio_set_dir(MS3_PIN, GPIO_OUT);
    gpio_set_dir(MS2_PIN, GPIO_OUT);
    gpio_set_dir(MS1_PIN, GPIO_OUT);

    // Turn them off
    gpio_put(STP_PIN, 0);
    gpio_put(DIR_PIN, 0);
    gpio_put(MS3_PIN, 0);
    gpio_put(MS2_PIN, 0);
    gpio_put(MS1_PIN, 0);
}

```

```

//
// Set Microstepping
// @brief Set the microstepping fraction and
// calculate the step delay for the input rpm
//
// @detail Assume in full step mode the motor is 1.8° per step,
// 1 revolution is 200 steps
//
// ms [in] - the microstepping mask
// rpm [in] - the revolutions per minute
//
// return - step period for input rpm
uint32_t set_microsteps(Microstepping_mask_t ms, float rpm) {
    uint32_t us_delay = 1; // min duration for step pulse high

    switch (ms) {
        case FULL_STEP :
            us_delay = (uint32_t) (1000000.0 * 60.0 / 200.0 / rpm);
            break;
        case HALF_STEP :
            us_delay = (uint32_t) (1000000.0 * 60.0 / 400.0 / rpm);
            break;
        case QUARTER_STEP :
            us_delay = (uint32_t) (1000000.0 * 60.0 / 800.0 / rpm);
            break;
        case EIGHTH_STEP :
            us_delay = (uint32_t) (1000000.0 * 60.0 / 1600.0 / rpm);
            break;
        case SIXTEENTH_STEP :
            us_delay = (uint32_t) (1000000.0 * 60.0 / 3200.0 / rpm);
            break;
        default: // FULL_STEP assumed
            us_delay = (uint32_t) (1000000.0 * 60.0 / 200.0 / rpm);
            break;
    }

    // Set all the GPIOs at once
    gpio_clr_mask(ALL_MS);
    gpio_set_mask(ms);

    // can't have the speed too high, the delay between steps too
    // short, not practical
    if (us_delay < 10) us_delay = 10;

    return us_delay;
}

void step(Direction_t direction, Microstepping_mask_t ms, uint16_t steps, float rpm) {
    gpio_put(DIR_PIN, 1);
    switch (direction) {
        case CW:
            gpio_put(DIR_PIN, 1);
            break;
        case CCW:
            gpio_put(DIR_PIN, 0);
            break;
        default:
            gpio_put(DIR_PIN, 0);
            break;
    }

    // Set the delay based on the desired rpm.
    // Account for the duration of stepping pulse
    uint32_t step_delay = set_microsteps(ms, rpm) - 10;

    for (uint16_t i = 0; i < steps; i++) {
        gpio_put(STP_PIN, 1);
        sleep_us(5); // min high time is 1 µs
        gpio_put(STP_PIN, 0);
        sleep_us(5); // min high time is 1 µs
        // speed delay
        sleep_us(step_delay);
    }
}

```

```
// Main Program
int main(void) {
    stdio_init_all();

    printf("A4988 driver example!\n");

    initialize_gpios();

    uint16_t steps = 200;

    float rpm = 30.0;

    while(1) {
        printf("200 Steps, FULL rpm = %f\n",rpm);
        step(CW, FULL_STEP, steps, rpm);
        sleep_ms(1000);
        step(CCW, FULL_STEP, steps, rpm);
        sleep_ms(1000);

        printf("200 Steps, HALF rpm = %f\n",rpm);
        step(CW, HALF_STEP, steps, rpm);
        sleep_ms(1000);
        step(CCW, HALF_STEP, steps, rpm);
        sleep_ms(1000);
    }
}
```

```
        printf("200 Steps, QUARTER rpm = %f\n",rpm);
        step(CW, QUARTER_STEP, steps, rpm);
        sleep_ms(1000);
        step(CCW, QUARTER_STEP, steps, rpm);
        sleep_ms(1000);

        printf("200 Steps, EIGHTH rpm = %f\n",rpm);
        step(CW, EIGHTH_STEP, steps, rpm);
        sleep_ms(1000);
        step(CCW, EIGHTH_STEP, steps, rpm);
        sleep_ms(1000);

        printf("200 Steps, SIXTEENTH rpm = %f\n",rpm);
        step(CW, SIXTEENTH_STEP, steps, rpm);
        sleep_ms(1000);
        step(CCW, SIXTEENTH_STEP, steps, rpm);
        sleep_ms(1000);

        // Reset or double the speed of rotation
        if (rpm >= 480.0)
            rpm = 30.0;
        else
            rpm *= 2.0;
    }
    return 0;
}
```

Next Lecture

- Loop control
- Feedback control

Next Lecture

- ~~Loop control~~
- ~~Feedback control~~
- Monday's class cancelled
- Wednesday is Garmin guest lecture
- April 3, 6 holidays
- May have lecture ready for the 8th
- April 10th open Q&A session

Questions

